

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	July 22, 2026	DATE SUBMITTED:	July 29, 2026
NAME:	EUNICE E. MORADILLO	INSTRUCTOR:	ENGR. JIMLORD M. QUEJADO

Activity 3.1	Linked Lists
1. PROCEDURE OUTPUT	

Screenshot

```
#include <iostream>
#include "singly_ll.h"

int main(){
    //creation of nodes "J,I,M,Q"

    //creating a node stored in the stack
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;

    //allocates memory in the heap
    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;

    //STORING A VALUE INSIDE THE NODE
    head->data = 'C';
    second->data = 'P';
    third->data = 'E';
    fourth->data = 'I';

    //LINKING THE NODES:
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = nullptr;

    std::cout << "Testing Traversal\n";
    ListTraversal(head);

    std::cout << "Testing of sllinsertHead:\n";
    sllinsertHead('G', &head);
    ListTraversal(head);

    std::cout<<"Testing of sllGeneralInsert:\n";
    sllGeneralInsert('E', head->next->next);
    ListTraversal(head);

    std::cout<<"Testing of sllinsertEnd:\n";
    sllinsertEnd('0', &head);
    ListTraversal(head);

    std::cout<<"Testing of deleting the node:\n";
    sllDeleteNode('C', &head);
    ListTraversal(head);
    sllDeleteNode('P', &head);
    ListTraversal(head);

    std::cout<<"Deleting all node:\n";
    sllDeleteList(&head);
    ListTraversal(head);

    return 0;
}
```

Discussion

This program is a singly linked list with four nodes that

	has the letters C, P, E, and 1. It tests each list function one by one, like inserting at the head, inserting in the middle, inserting at the end, and deleting a node, then prints the list after every step to check if the function worked. Running it this way helped me confirm that each operation correctly updates the pointers without breaking the list.
--	--

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	<pre> template <typename T> void ListTraversal(SingleList<T>* head){ if (head == nullptr){ std::cout << "The list is empty"; return; } while(head != nullptr){ // PRINT data OF n std::cout<<head->data; if(head->next != nullptr){ std::cout << "->"; } // GO TO NEXT NODE n -> next head = head->next; } std::cout << std::endl; } </pre>
Insertion at head	<pre> template <typename T> void sllinsertHead(T newData, SingleList<T>** head){ // 1. Allocate new memory for the new code SingleList<T>* newNode = new SingleList<T>; // 2. Put our data into the new node newNode->data = newData; // 3. Set next to the new node to point to the previous newNode->next = *head; // 4. Reset head to the new node *head = newNode; } </pre>

Insertion at any part of the list

```
template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* prevNode){

    //1. Check if it is the head node (previous node is null)
    if(prevNode == nullptr){
        //2. If null, print "Previous node cannot be null."
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    //3. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
    //4. Store data in the new node
    newNode->data = newData;
    //5. Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    //6. Point previous node to the new node
    prevNode->next = newNode;
}
```

Insertion at the end

```
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    // 1. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;

    // 3. Store data in new node
    newNode->data = newData;
    // 4. Point to null
    newNode->next = nullptr;

    // 2. Dereference the head node
    SingleList<T>* currentNode = *head;

    // 5. Traverse the list
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    // 6. Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

Deletion of a node

```
template <typename T>
void sllDeleteNode(T findData, SingleList<T>** head){

    // if there is nothing to delete
    if (*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    // searching the node: 1. the node does not exist + we found data
    while (currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    // if data not found currNode == nullptr in the while loop
    if(currNode == nullptr){
        std::cout << "the data was not found\n" << std::endl;
        return;
    }

    // if data was found
    if(prevNode == nullptr){
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

Table 3-2. Code for the List Operations

a.	Source Code	<pre>std::cout << "Testing Traversal\n"; ListTraversal(head);</pre>
	Console	<pre>Testing Traversal C->P->E->1</pre>
b.	Source Code	<pre>std::cout << "Testing of sllinsertHead:\n"; sllinsertHead('G', &head); ListTraversal(head);</pre>
	Console	<pre>Testing of sllinsertHead: G->C->P->E->1</pre>
c.	Source Code	<pre>std::cout<<"Testing of sllGeneralInsert:\n"; sllGeneralInsert('E', head->next->next); ListTraversal(head);</pre>
	Console	<pre>Testing of sllGeneralInsert: G->C->P->E->E->1</pre>
d.	Source Code	<pre>std::cout<<"Testing of sllinsertEnd:\n"; sllinsertEnd('0', &head); ListTraversal(head);</pre>
	Console	<pre>Testing of sllinsertEnd: G->C->P->E->E->1->0</pre>
e.	Source Code	<pre>std::cout<<"Testing of deleting the node:\n"; sllDeleteNode('C', &head); ListTraversal(head); sllDeleteNode('P', &head); ListTraversal(head);</pre>
	Console	<pre>Testing of deleting the node: G->P->E->E->1->0 G->E->E->1->0</pre>
f.	Source Code	<pre>std::cout<<"Deleting all node:\n"; sllDeleteList(&head); ListTraversal(head);</pre>
	Console	<pre>G->E->E->1->0</pre>

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots(s)	Analysis
----------------	----------

```
template <typename T>
void dllTraverse(DoubleList<T>* currentNode){
    //check if the list is empty
    if(currentNode == nullptr){
        std::cout<<"The list is empty";
        return;
    }
    DoubleList<T>* tail;
    std::cout<<"FORWARD"<<std::endl;
    //forward traverse
    while(currentNode != nullptr){
        std::cout<<currentNode->data << " ";
        tail = currentNode;
        currentNode=currentNode->next;
    }
    std::cout<<std::endl;
    //reverse traverse
    std::cout<<"REVERSE"<<std::endl;
    while (tail != nullptr){
        std::cout<<tail->data <<" ";
        tail = tail -> prev;
    }
    std::cout<<std::endl;
}
```

This function prints the list both forward and backward, using the next pointers going forward and the prev pointers going backward. It stores the tail while going forward so it knows where to start the reverse traversal. This shows the doubly linked list which is being able to move both directions.

```
//inserting a node to the head
template <typename T>
void dllInsertHead(T newData, DoubleList<T>** currentNode){

    //create a new node
    DoubleList<T> *newNode = createNode(newData);
    //point the newNode to the currentNode (head)
    newNode->next = *currentNode;
    //point the currentNode to the newNode
    if (*currentNode != nullptr){
        (*currentNode)->prev = newNode;
    }
    //updating the header pointer
    *currentNode = newNode;
}
```

This function creates a new node and links it to the current head using both the next and prev pointers. It checks if the current head exists before updating its prev pointer, so it won't crash on an empty list. This keeps the list correctly connected in both directions after insertion.

```
//general insert
template <typename T>
void dllGenInsert(DoubleList<T>* prevNode, T newData) {
    if (prevNode == nullptr) {
        std::cout << "Cannot insert, target node is null!" << std::endl;
        return;
    }
    DoubleList<T>* newNode = new DoubleList<T>;
    newNode->data = newData;
    newNode->next = prevNode->next;
    newNode->prev = prevNode;
    if (prevNode->next != nullptr){
        prevNode->next->prev = newNode;
    }
    prevNode->next = newNode;
}
```

this function inserts a new node right after a given previous node, updating four pointers instead of two like in a singly linked list. It also checks if the previous node is null before doing anything, to avoid inserting into nothing. This shows that doubly linked lists need more pointer updates per operation, but allow easier traversal later.

```
//inserting at the End;
template <typename T>
void dllInsertEnd (T newData, DoubleList<T> *currentHead){
    //create a new node
    DoubleList<T>* newNode = createNode(newData);
    //traverse until we reach the last node
    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }
    //connect the new node back to the last node
    newNode->prev = currentHead;
    //connect the current node to the new node
    currentHead->next = newNode;
}
```

These functions traverse the list until they reach the last node, then attach the new node there and link it back with the "prev" pointer. Since both versions do the same thing, this shows a repeated/duplicate function in the code that could be cleaned up. Still, the logic correctly maintains both forward and backward links at the end of the list.

```
//delete a node by its data value
template <typename T>
void dllDeleteNode(T data, DoubleList<T>** head){
    DoubleList<T>* curr = *head;
    while (curr != nullptr){
        if (curr->data == data){
            dllDelete(head, curr);
            return;
        }
        curr = curr->next;
    }
}
```

This function searches the doubly linked list for the first node whose `data` matches the given value, then deletes that node. If no match is found, it does nothing.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
#include "doubly_ll.h"

int main(){

    Song* playlist = nullptr;

    //create a playlist using multiple songs
    addSong("Song A", &playlist);
    addSong("Song B", &playlist);
    addSong("Song C", &playlist);
    addSong("Song D", &playlist);

    std::cout << "Full playlist:" << std::endl;
    playAll(playlist);

    //add a song to the playlist
    std::cout << "\nAdding 'Song E':" << std::endl;
    addSong("Song E", &playlist);
    playAll(playlist);

    //remove a song from the playlist
    std::cout << "\nRemoving 'Song C'" << std::endl;
    removeSong("Song C", &playlist);
    playAll(playlist);

    //demonstrate next and previous from the current song
    Song* current = playlist;
    std::cout << "\nCurrent: " << current->title << std::endl;

    current = nextSong(current);
    std::cout << "Next: " << current->title << std::endl;

    current = nextSong(current);
    std::cout << "Next: " << current->title << std::endl;

    current = prevSong(current);
    std::cout << "Previous: " << current->title << std::endl;

    //delete the whole playlist
    std::cout << "Deleting playlist" << std::endl;
    deletePlaylist(&playlist);
    playAll(playlist);

    return 0;
}
```

MAIN CPP FILE

```

1  ▾ #ifndef DOUBLY_LL_H
2  ▾ #define DOUBLY_LL_H
3
4  ▾ #include <iostream>
5
6  ▾ // a song holds its title and links to the song before and after it
7  ▾ class Song{
8  ▾ public:
9  ▾     std::string title;
10 ▾     Song *next;
11 ▾     Song *prev;
12 ▾ };
13
14 ▾ // adds a new song to the end of the playlist
15 ▾ void addSong(std::string title, Song **playlist){
16 ▾     Song *newNode = new Song;
17 ▾     newNode->title = title;
18
19 ▾     if(*playlist == nullptr){
20 ▾         // playlist was empty, so this new song becomes the only song
21 ▾         newNode->next = newNode; // points to itself since it is circular
22 ▾         newNode->prev = newNode;
23 ▾         *playlist = newNode;
24 ▾     } else {
25 ▾         Song *head = *playlist;
26 ▾         Song *tail = head->prev;
27
28 ▾         // link the new song in between the tail and the head
29 ▾         newNode->prev = tail;
30 ▾         newNode->next = head;
31 ▾         tail->next = newNode;
32 ▾         head->prev = newNode;
33 ▾     }
34 ▾ }
35
36 ▾ // removes a song from the playlist by title
37 ▾ void removeSong(std::string title, Song **playlist){
38 ▾     if(*playlist == nullptr) return;
39
40 ▾     Song *temp = *playlist;
41 ▾     do{
42 ▾         if(temp->title == title){
43 ▾             if(temp->next == temp){
44 ▾                 // only one song was in the list
45 ▾                 *playlist = nullptr;
46 ▾             } else {
47 ▾                 // skip over the node being removed
48 ▾                 temp->prev->next = temp->next;
49 ▾                 temp->next->prev = temp->prev;
50 ▾                 if(*playlist == temp) *playlist = temp->next;
51 ▾             }
52 ▾             delete temp; // free the memory
53 ▾             return;
54 ▾         }
55 ▾         temp = temp->next;
56 ▾     } while(temp != *playlist);
57 ▾ }

```



```

57 }
58
59 // prints every song in the playlist separated by a dash
60 void playAll(Song *playlist){
61     if(playlist == nullptr){
62         std::cout << "Playlist is empty" << std::endl;
63         return;
64     }
65     Song *temp = playlist;
66     do{
67         std::cout << temp->title;
68         if(temp->next != playlist) std::cout << " - ";
69         temp = temp->next;
70     } while(temp != playlist);
71 }
72
73 // moves to the next song
74 Song* nextSong(Song *current){
75     return current->next;
76 }
77
78 // moves to the previous song
79 Song* prevSong(Song *current){
80     return current->prev;
81 }
82
83 // deletes every song in the playlist and frees the memory
84 void deletePlaylist(Song **playlist){
85     if(*playlist == nullptr) return;
86
87     Song *current = *playlist;
88     Song *temp;
89     do{
90         temp = current;
91         current = current->next;
92         delete temp;
93     } while(current != *playlist);
94
95     *playlist = nullptr;
96 }
97
98 #endif
99

```

HEADER FILE

OUTPUT

ANALYSIS:

In this code, it composes of a main.cpp file and a header file to make a linked list to manage a song playlist, where as in each song points to the next and previous song and connects the last song back to the first. The addsong functions adds a new song by linking the tail and head pointer. The removeSong function will find a song that will be unlinked. The playAll, nextSong, and prevSong functions display the playlist by moving forward and backward in between the songs. The deletePlaylist will delete all. This program overall is structured to handle double list like updating the head pointer when the first song is removed and stopping loops at the right point instead of looping forever.

CONCLUSION:

Through this activity, I learned how singly and doubly linked lists work, especially how pointers connect and disconnect nodes during insertion and deletion. The procedure helped me see that singly linked lists only move forward, while doubly linked lists let you move both ways using the prev pointer. The supplementary activity applying a circular doubly linked list to a song playlist showed me how these same operations can be used in a real program, like skipping to the next or previous song. I think I did well in understanding and testing the functions, but I still need to improve on writing cleaner code and avoiding duplicate functions like the two versions of dllInsertEnd (I had to redo and redo this because the file wont save).

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Microsoft copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of concepts, wordings

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Used AI in the google search bar to help me explain my explanation be better like synonyms and how to grammatically say this right.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.